



MERRIMACK COLLEGE

CSC 6033

Week 1

Course Introduction and From Python to C++

Languages, Automata, and Decidability - Dr. Paulo Fernandes

Presentation Agenda

Week 1

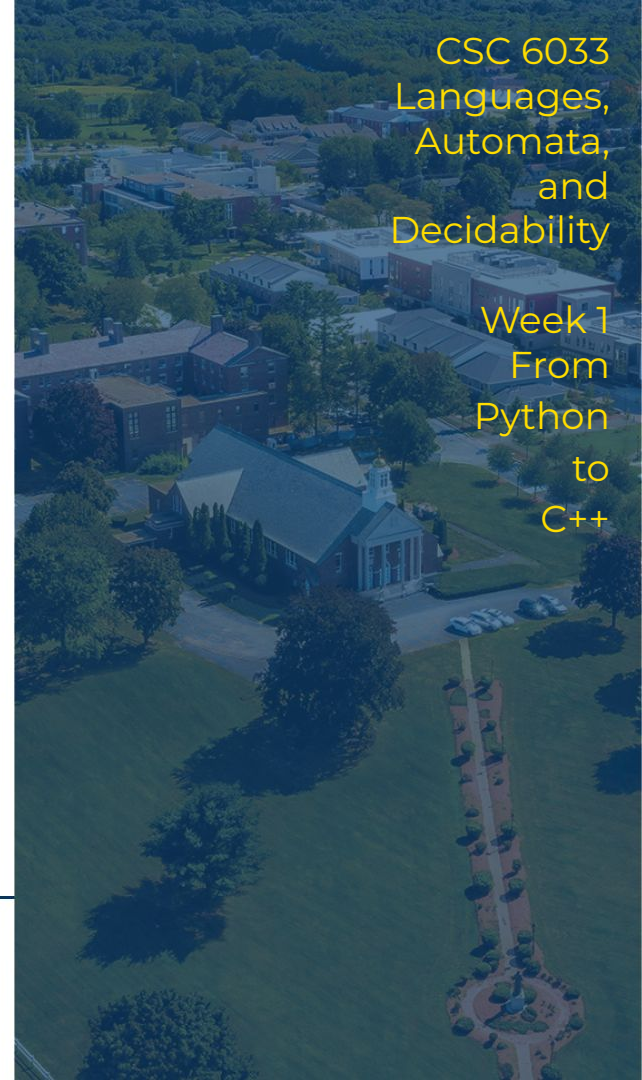
- Course Introduction
 - a. Course format
 - b. Evaluation
 - c. Timeline
- From Python to C++
 - a. The C++ language
 - b. Installing C/C++, Hello World!
 - c. Simple C++ programs
- This Week's tasks



MERRIMACK COLLEGE

CSC 6033
Languages,
Automata,
and
Decidability

Week 1
From
Python
to
C++



Course Introduction

We will introduce C++ and
the basics of Theory of Computation.

Welcome!

This course is entitled CSC6033
Languages, Automata, and
Decidability.

This course comes just after the
Program's Core Courses (CSC6003,
CSC6013, and CSC6023). As such, it can
be considered an Advanced Course,
but it could also be seen as the
conclusion of the Computer Scientist
basic formation.

- Course format
- Timeline
- Evaluation



Course Format

This course is fully online, all weeks will have:

- Live sessions every Tuesdays 6:30pm to 8:30pm
- Office Hours every Thursday 8:30pm to 9:30pm

Every week, but the last, we will have:

- Discussion/In-class exercises tasks until Saturday
- Quizzes to be taken from Saturday to Next Tuesday
- Coding assignments to be turned in until Next Tuesday

Last week has:

- Final exam to be done until Saturday

Communication

- fernandesf@merrimack.edu



MERRIMACK COLLEGE

Zoom chat per request

Course Format

- Live sessions every Tuesdays 6:30pm to 8:30pm
 - Meeting ID: 999 3460 4558 Passcode: CSC6033
- Office Hours every Thursdays 8:30pm to 9:30pm
 - Meeting ID: 999 3460 4558 Passcode: CSC6033
- In-class exercises tasks until Saturday
 - Simple exercises, and/or posts on the discussion board
- Quizzes to be taken from Saturday to Next Tuesday
 - Ten multiple choice questions, open book, open notes, and untimed
- Coding assignments to be turned in until next Tuesday
 - Programming tasks
- Final exam to be done until last Saturday of classes
 - 8 questions, 4 hours to take it, open book, open notes



Timeline

This is a very fast-paced course (better get used to).

Try to check it out the topics before the live session.

Any change will be informed in class and on Canvas.

Unit	Week	Topic	Projects	Quizzes	In-class Exercises	Exam
1	1	From Python to C++	P#1	Q#1	E#1	
1	2	Developing with C++	P#2	Q#2	E#2	
2	3	Finite Automata	P#3	Q#3	E#3	
2	4	Regular Expressions	P#4	Q#4	E#4	
3	5	Pushdown Automata	P#5	Q#5	E#5	
4	6	Turing Machines	P#6	Q#6	E#6	
5	7	Processing Input and Output	P#7	Q#7	E#7	
6	8	Decidability				Final Exam (all units)



MERRIMACK COLLEGE

Don't let tasks pile up!

Evaluation

The evaluated tasks are weighted as such (105%):

Activity	Published grades	Percentage
Projects (7)	Evaluated from 0 to 100 each	35.0%
Quizzes (7)	Evaluated from 0 to 100 each	28.0%
In-class Exercises (7)	Evaluated from 0 to 100 each	21.0%
Final Exam	Evaluated from 0 to 100	21.0%

All tasks are due in their stated deadline, the late penalties reduced 10% of the evaluation, plus 2% for each full day of delay.



MERRIMACK COLLEGE

Final deadline on Friday of the course's last week

Evaluation

The final letter grade is computed according to this table:

A	A-	B+	B	B-	C+	C	C-	F
95 and up	90 to 94.9	87 to 89.9	83 to 86.9	80 to 82.9	77 to 79.9	73 to 76.9	70 to 72.9	69.9 and low

There are no grade D+, D, or D- in graduate courses. Only C or above are passing grades. An average grade B is required to the Masters of Science in Computer Science, lower than that puts you in probation.



MERRIMACK COLLEGE

Final grades published first Monday after the course's end.

From Python to C++

C++ language is the first broadly used programming language providing Object-Oriented concepts.

High-level programming languages is nowadays a deprecated denomination. Yet, in the 70's the C language became the high-level programming language to develop.

It was efficient, concise and an unavoidable paradigm for programming languages. A little more than a decade after came C++.

- **The C++ language**
- Installing C/C++, Hello World!
- Simple C++ programs



The C++ language



The **C++** programming language is the main derivative language of the **C** programming language.

In 1972 within Bell Labs, *Dennis Ritchie* designed a programming language with efficiency (fast to run) and ergonomicity (fast to type) in mind. This **C** language was mostly intended to code routines for Unix Operating System.

After finishing his PhD, *Bjarne Stroustrup* start in 1979 to design a programming language extension to **C** to include object oriented concepts. He decided to called "**C with classes**". In 1982 he decided to rename the language to **C++** since it was the language **C** incremented (**++** after an Integer variable in **C** means to increment the content of this variable). This was a debatable choice for the time, since up to the end of 90's web browsers did not allowed search with **+** signs.



C++ language is a superset of **C**.
Every **C** program is also a **C++** program.

History and Purpose

				
Creation and big revisions	1972 1990, 1999 , 2017	1985 1998 , 2011	1995 2006, 2017...	1991 2000, 2008
Initial Purpose	Simple code	Add OO to C	Bring OO to the industry	Solve practical problems fast
Strongest Suit	Very close to hardware	Performance and flexibility	Facilitate code reuse	Fast prototyping
Market Share	Embedded Systems	Classical CS, Academia	Big industry, Large projects	Open source, Artificial Intel.
<i>Trend</i>	<i>Stable</i>	<i>Stable</i>	<i>Declining</i>	<i>Growing</i>



MERRIMACK COLLEGE

Wikipedia: [C](#) - [C++](#) - [Java](#) - [Python](#)

From Python to C++

C++ language is the first broadly used programming language providing Object-Oriented concepts.

High-level programming languages is nowadays a deprecated denomination. Yet, in the 70's the C language became the high-level programming language to develop.

It was efficient, concise and an unavoidable paradigm for programming languages. A little more than a decade after came C++.

- The C++ language
- **Installing C/C++, Hello World!**
- Simple C++ programs



Running code online

Among the several online options you can try these ones:

You can try it on one of the following C++ interpreters online:

[OnlineGDB](#) - [W3S](#) - [CPPsh](#)

Likely your machine already has C++ installed for Operating System sake.

However, for small C++ programs it is just as easy to use online servers.



A curious thing is that such online servers always behave as interpreters, but refer themselves as compilers, probably due to their linking functionalities.



MERRIMACK COLLEGE

Friendly advice, using an online server as such, always keep a local copy of your code.

Installing C++ in your machine

- Choose an IDE
 - There are several options, I personally like Visual Studio Code that is free, available to all Operating Systems, and reasonably light compared to other options
 - VSCode official page: <https://code.visualstudio.com>
- Install the proper compiler according to your Operating System (Windows, MacOS, or Linux)
 - Video exemplifying the setup C++ in a Windows system [here](#)
 - Video exemplifying the setup C++ in a MacOS system [here](#)



Running a program in Python and in C++

- An interpreted language - like Python
 - Edit and run your program:
 - Your IDE calls the Python interpreter and pass your code to it;
 - Each command of your program is executed by the interpreter;
 - The input and output is handled by the interpreter.
- A compiled language - like C++
 - Edit and run your program:
 - Your IDE compiles each module of your program
 - (it generates obj code pieces, usually optimizing);
 - The generated obj code pieces are linked
 - (it generates an executable file);
 - The executable file is executed by the OS.



The Hello World in C++

Let's see it on [OnlineGDB](#):

- The input and output is done, usually, using the package ***iostream***;
- Every **C++** program, just like **C**, starts with a function `main()` that has an ***int*** type:
 - if the program executes as expected it returns ***0***, otherwise it returns an Integer error code (like ***1***, for example);
- The output is made by the default output stream ***cout*** that is defined within the package `iostream` in the namespace ***std***;
- The string **"Hello World"** is sent to the standard output (***std::cout***);
- The ***return 0*** informs that the function ***main*** was successfully ended.

```
1  #include <iostream>
2  int main() {
3      std::cout << "Hello World!";
4      return 0;
5  }
```

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      cout << "Hello World!";
5      return 0;
6  }
```



In a nutshell

```
1 import math
2
3 def is_prime(n):
4     if n <= 1:
5         return False
6     for i in range(2, int(math.sqrt(n)) + 1):
7         if n % i == 0:
8             return False
9     return True
10
11 print(is_prime(31)) # True
12 print(is_prime(39)) # False
```

Blocks by
indentation



In Python

- Mostly packages, sometimes scripts, sometimes a main function, sometimes classes;

single line remarks

```
1 #include <iostream>
2 #include <cmath>
3
4 bool is_prime(int n) {
5     if (n <= 1) {
6         return false;
7     }
8     for (int i = 2; i <= sqrt(n); ++i) {
9         if (n % i == 0) {
10            return false;
11        }
12    }
13    return true;
14 }
15
16 int main() {
17     std::cout << is_prime(31) << std::endl; // True
18     std::cout << is_prime(39) << std::endl; // False
19     return 0;
20 }
```

Blocks by
curly braces



In C++

- Packages, a main function calling everything else, and frequently classes.

single line
remarks



MERRIMACK COLLEGE

Video: [Learn C++ in 10 Minutes !!](#).

From Python to C++

C++ language is the first broadly used programming language providing Object-Oriented concepts.

High-level programming languages is nowadays a deprecated denomination. Yet, in the 70's the C language became the high-level programming language to develop.

It was efficient, concise and an unavoidable paradigm for programming languages. A little more than a decade after came C++.

- The C++ language
- Installing C/C++, Hello World!
- **Simple C++ programs**



Basic Commands - variables

- In C++
 - Integers and Floating points (different precisions), Pointers, Booleans, Chars, Enumerations, Arrays, and Strings;
 - Declaration and allocation needed;
- In Python
 - Integers, Floating points, Booleans, and Lists (including Strings);
 - No declaration or allocation needed

```
long int a = 1048356;  
double arr[10];  
bool tst = true;  
enum = { 'm', 't', 'w', 'h', 'f' }
```



```
fact50 = 30414093201713378043612608166064768844377641568960512000000000000  
  
print(10/3)  
  
3.3333333333333335
```

```
myList = [ 0.5, "half", [1, "divided by", 2]
```



Basic Commands - decision

- In Python
 - if - elif - else
 - shorthand if-else
- In C++
 - if - else
 - conditional operator
 - switch - case - default



```
if (a < b):  
    print("a is smaller than b")  
elif (a > b):  
    print("a is greater than b")  
else:  
    print("a and b are equal")
```

```
print("a and b are equal") if (a == b) else print("a and b are different")
```

```
switch(a) {  
    case 12:  
        std::cout << "a is a dozen" << std::endl;  
        break;  
    case 13:  
        std::cout << "a is bad luck" << std::endl;  
        break;  
    default:  
        std::cout << "a is something else" << std::endl;  
}
```



```
if (a < b) {  
    std::cout << "a is smaller than b" << std::endl;  
}  
else {  
    if (a > b) {  
        std::cout << "a is greater than b" << std::endl;  
    }  
    else {  
        std::cout << "a and b are equal" << std::endl;  
    }  
}
```

```
(a == b) ? std::cout << "a and b are equal" << std::endl : std::cout << "a and b are different" << std::endl;
```



Basic Commands - loops

- In Python
 - for loop
 - range - break - continue - pass - else
 - while loop
 - break - continue - else
- In C++
 - for loop
 - break - continue - goto
 - while loop
 - break - continue - goto
 - do while loop
 - break - continue - goto

```
for i in range(5):  
    print(i+1)
```

```
i = 1  
while (i <= 5):  
    print(i)  
    i += 1
```



```
for (int i=0; i<5; i++) {  
    std::cout << i+1 << std::endl;  
}
```

```
i = 1;  
do {  
    std::cout << i << std::endl;  
    i++;  
} while (i <= 5);
```

```
int i = 1;  
while (i <= 5) {  
    std::cout << i << std::endl;  
    i++;  
}
```



Basic Commands - functions

- In Python
 - functions are untyped
 - input parameters can be omitted/default values
 - simple variables passed by value, lists passed by reference (almost always)
 - none, single, or multiple return values/objects
- In C++
 - functions are explicitly typed (including `void`)
 - function overload on different parameter sets
 - input parameters passed either by value or reference (programmer choice)
 - none or single return value/object



```
def myKids(*kids):  
    print("The youngest kid is", kids[-1])  
    print("The eldest kid is", kids[0])  
    print("I have", len(kids), "kids")  
  
myKids("Karina", "Maria", "Sophia")
```



```
#include <iostream>  
using namespace std;  
  
void myName(string);  
  
int main() {  
    myName("Paulo");  
    return 0;  
}  
  
void myName(string name) {  
    cout << "My name is " << name << endl;  
}
```



An Example - Matrix Multiplication

- A driver that creates the two matrices as a double dimension pointer, plus its number of rows and columns;
- ... and call the ***printProduct*** function to perform the multiplication and print the result.

```
#include <iostream>
using namespace std;

> void printProduct(int** mat1, int** mat2, int r1, int c1, int r2, int c2) ...

// Driver code
int main()
{
    int r1 = 2, c1 = 2;
    int r2 = 2, c2 = 3;
    int** mat1 = new int*[r1];
    for (int i=0; i<r1; i++) {
        mat1[i] = new int[c1]; }
    mat1[0][0] = 1;   mat1[0][1] = 2;
    mat1[1][0] = 3;   mat1[1][1] = 4;
    int** mat2 = new int*[r2];
    for (int i=0; i<r2; i++) {
        mat2[i] = new int[c2]; }
    mat2[0][0] = 10;  mat2[0][1] = 30;  mat2[0][2] = 50;
    mat2[1][0] = 20;  mat2[1][1] = 40;  mat2[1][2] = 60;
    printProduct(mat1, mat2, r1, c1, r2, c2);
    return 0;
}
```



	1	2			10	30	50	
	3	4			20	40	60	

	50	110	170	
	110	250	390	



An Example - Matrix Multiplication

- A function to multiply and print two matrices received each as:
 - a double dimension pointer, plus its number of rows and columns.

	1	2				10	30	50	
	3	4				20	40	60	

Multiplication of given two matrices is:

50	110	170
110	250	390

```
void printProduct(int** mat1, int** mat2, int r1, int c1, int r2, int c2)
{
    if (c1 != r2) {
        cout << "Impossible: The number of columns in the first matrix is "
              << c1 << " and the number of rows in the second matrix is "
              << r2 << endl;
    }
    else {
        int rslt[r1][c2];
        cout << "Multiplication of given two matrices is:\n";
        for (int i=0; i<r1; i++) {
            for (int j=0; j<c2; j++) {
                rslt[i][j] = 0;
                for (int k=0; k<c1; k++) {
                    rslt[i][j] += mat1[i][k] * mat2[k][j];
                }
                cout << rslt[i][j] << "\t";
            }
            cout << endl;
        }
    }
}
```



dot product

$$\begin{bmatrix} 1 & 2 \end{bmatrix} \times \begin{bmatrix} 10 & 20 \end{bmatrix} = 1 \times 10 + 2 \times 20 = 50$$



MERRIMACK COLLEGE

Video: [Matrix Multiplication](#).

This Week's tasks

- In-class Exercise E#1
- Coding Project P#1
- Quiz Q#1

Tasks

- Perform the matrix multiplication of two examples and instrument the code to print the vectors to be multiplied.
- Create your own C++ program to compute the dot product of two vectors.
- Quiz #1 about this week topics.



In-class Exercise - E#1

Perform the matrix multiplication of examples **a** and **b** and show the vectors to be multiplied at each dot product step (refer to the code in slides 23 and 24).

- Every time that you need to perform a dot product, show the two vectors and the dot product numerical result;
- For the example given in class, you should print:

1	2		10	30	50	
3	4		20	40	60	
- You can either edit the code shown in class to insert the desired output, or you can trace the code performing the execution manually.

The examples

a)

2	5		6	9	11	13	
3	8		12	18	22	26	

b)

1	2	4		3	5	
8	16	32		7	9	
64	128	256		11	13	

[1 2]	x	[10 20]	=	50
[1 2]	x	[30 40]	=	110
[1 2]	x	[50 60]	=	170
[3 4]	x	[10 20]	=	110
[3 4]	x	[30 40]	=	250
[3 4]	x	[50 60]	=	390

You have to submit a **.pdf** file with the output for the examples **a** and **b** above.



MERRIMACK COLLEGE

This task counts towards the In-class Exercises grade and the deadline is This Saturday.

First Coding Project - P#1

- Create your own C++ program to compute the dot product of two vectors:
 - Your program should be composed by **a function that performs the dot product of two vectors** received each as a single dimension pointer, plus its number of elements of both vectors.
 - This function should receive three input parameters: the first vector (a pointer to a vector of Integers), the second vector (a pointer to a vector of Integers), and the number of elements for both vectors.
 - This function should return the dot product numerical result.
 - Your program should have driver (**the function main**) to create the vectors and call the dot product function and print out the result.

Your task:

- Go to Canvas, and submit your C++ file (.cpp) within the deadline.



MERRIMACK COLLEGE

This assignment counts towards the Projects grade and the deadline is Next Tuesday.

First Coding Project - P#1

- For example, your driver can define the vectors **[8 16 32]** and **[5 9 13]** and it should print the output the result 600;
- Another example, your driver can define the vectors **[3 8]** and **[11 22]** and it should print the output the result 209;
- Feel free to be inspired by the example shown to perform matrix multiplication, or even search for examples in the Internet;
 - However, make sure you fully understand the examples you use as learning source, and
 - The produced code must be your own - do not copy, neither retype seeing the source.
 - If you need to add any information make it in the code remarks.
 - The documentation is not required, but include in your code your name.



MERRIMACK COLLEGE

This assignment counts towards the Projects grade and the deadline is Next Tuesday.

First Quiz - Q#1

- The first quiz in this course covers the topics of Week 1;
- The quiz will be available this Saturday, and it is composed by 10 questions;
- The quiz should be taken on Canvas (Module 1), and it is not a timed quiz:
 - You can take as long as you want to answer it;
- The quiz is open book, open notes, and you can even use any language Interpreter to answer it;
- Yet, the quiz is evaluated and you are allowed to submit it only once.

Your task:

- Go to Canvas, answer the quiz and submit it within the deadline.



MERRIMACK COLLEGE

This quiz counts towards the Quizzes grade and the deadline is Next Tuesday.

” **Welcome to CSC 6033**

- **Read the syllabus**
 - **Do In-class Exercise E#1 until Saturday;**
 - **Do Quiz Q#1 (available Saturday) until next Tuesday;**
 - **Do Coding Project P#1 until next Tuesday.**
-

Next Week - Developing with C++



MERRIMACK COLLEGE